

1. Quick Sort in Python

Problem Statement

Write a Python program that sorts an array using Quick Sort.

Aim

To implement Quick Sort and display the array before and after sorting.

Theory

Quick Sort is a divide-and-conquer sorting algorithm. It chooses a pivot, partitions the remaining values into smaller and larger groups, then recursively sorts those groups. The result is a new sorted list. Average time complexity is $O(n \log n)$; the worst case is $O(n^2)$.

Algorithm

1. If the input has fewer than two values, return it.
2. Choose the last value as the pivot.
3. Split the remaining values into those less than or equal to the pivot and those greater than it.
4. Recursively sort each group and join them around the pivot.

Code

```
def quick_sort(values):  
    if len(values) < 2:  
        return values  
    pivot = values[-1]  
    left = [value for value in values[:-1] if value <= pivot]  
    right = [value for value in values[:-1] if value > pivot]  
    return quick_sort(left) + [pivot] + quick_sort(right)  
  
numbers = [8, 3, 5, 1]  
print("Original:", numbers)  
print("Sorted:", quick_sort(numbers))
```

Expected Output (not runtime verified)

Illustrative expected output for this public sample. Run the code yourself before relying on it.

```
Terminal  
user@host:~$ python practical1.py  
Original: [8, 3, 5, 1]  
Sorted: [1, 3, 5, 8]
```

Conclusion

The sample array is arranged in ascending order using recursive partitioning.